arXiv:2106.12151v2 [cs.AI] 28 Oct 2021

Width-based Lookaheads with Learnt Base Policies and Heuristics Over the Atari-2600 Benchmark

Stefan O'Toole
Computing and Information Systems
University of Melbourne, Australia
stefan@student.unimelb.edu.au

Nir Lipovetzky
Computing and Information Systems
University of Melbourne, Australia
nir.lipovetzky@unimelb.edu.au

Miquel Ramirez
Electrical and Electronic Engineering
University of Melbourne, Australia
miquel.ramirez@unimelb.edu.au

Adrian R. Pearce
Computing and Information Systems
University of Melbourne, Australia
adrianrp@unimelb.edu.au

Abstract

We propose new width-based planning and learning algorithms inspired from a careful analysis of the design decisions made by previous width-based planners. The algorithms are applied over the Atari-2600 games and our best performing algorithm, Novelty guided Critical Path Learning (N-CPL), outperforms the previously introduced width-based planning and learning algorithms π-IW(1), π-IW(1)+ and π-HIW(n, 1). Furthermore, we present a taxonomy of the Atari-2600 games according to some of their defining characteristics. This analysis of the games provides further insight into the behaviour and performance of the algorithms introduced. Namely, for games with large branching factors, and games with sparse meaningful rewards, N-CPL outperforms π-IW, π-IW(1)+ and π-HIW(n, 1).

1 Introduction

The Atari-2600 games provide useful environments for benchmarking autonomous agents due to the diversity of behaviour required across the different games. The Atari-2600 games can be accessed through the Arcade Learning Environment (ALE) (1) which provides a typical Reinforcement Learning (RL) environment interface where given a state, the agent selects an action and receives a resulting state and reward. The two main approaches that have been used by autonomous agents applied to the Atari-2600 games have been RL methods (2; 3; 4) and Planning methods (5; 6). The RL approaches have had great success surpassing the performance of human players for many of the Atari-2600 games. However, RL approaches require long training times in order to train the Neural Networks (NN) used for policy and value functions. Planning agents do not require training time and instead use a bounded, fixed computational budget to decide which action to take at each time step of the game. The budget allowed for planning for each action is set as part of the experimental setting and can be set in such a way that the agent can play a game in real-time. Through the ALE interface, the agent is not provided a description of the transition or reward functions as is the case of models described through languages such as the Planning Domain Description Language (PDDL) (7). Instead, planning agents applied to the Atari-2600 games are required to work with a simulator, treating the environment's transition and reward functions as a black-box (5).

Width-based planning agents have been shown to be particularly successful on the Atari-2600 games when compared to other planning agents (5; 6). Width-based planners prioritise search effort on states deemed to be novel. The novelty of a state can be defined in a number of ways. Previously, novelty tests have been obtained from the RAM of the game (5), handcrafted features computed from screen

Preprint. Under review.

pixels (6) and learnt features extracted from the screen pixels through a NN (8; 9; 10). In this paper we consider planners with a novelty measure that does not require extensive feature engineering, or the internal state of the simulator, but is instead defined directly over the values of screen pixels.

Recent approaches have combined the RL and planning methods into single agents that are applied to the Atari-2600 games (8; 11; 10). Junyent et al. (8) combined a width-based planner with a learnt policy defined over a NN in order to guide the planner to promising areas of the search space. The learnt NN was also used to extract features from which the novelty of states were defined over. In this paper we introduce new width-based planning and learning methods that learn both policy and value networks using a methodical learning schedule.

Through analysing previous width-based methods we construct and benchmark new width-based approaches for the Atari-2600 games. We also classify the Atari-2600 games according to their particular characteristics. The resulting game taxonomy helps us to gain insight into the performance of the algorithms we propose and benchmark. The paper contributions are: (1) an analysis of the previous width-based planning methods that have been applied to the Atari-2600 games, (2) introducing new width-based planning and learning approaches for playing the Atari-2600 games, (3) defining a methodical learning schedule for planning and learning methods, and (4) identifying characteristics of the Atari-2600 games that influence the performance of different planning approaches.

2 Background

2.1 MDPs

We model the Atari games as Markov Decision Processes (MDPs). We formalise MDPs, as described by Geffner and Bonet (12), as the tuple of $M = (\mathcal{S}, s_0, A, T, R)$, where $\mathcal{S} \subseteq \mathbb{R}^d$, $s_0 \in \mathcal{S}$ is the initial state, A provides the sets of applicable actions such that $A(s)$ is a set of actions applicable in $s \in \mathcal{S}$, T is a set of distributions such that $T(s, a, s')$ gives the probability of the transition from state $s \in \mathcal{S}$ to state $s' \in \mathcal{S}$ given action $a \in A(s)$, and R is the reward function such that $R(s, a)$ returns the reward for performing action $a \in A(s)$ from state $s \in \mathcal{S}$. In this work we will be considering a special case, finite-horizon MDPs, where accumulated rewards need to be maximised over a given number of stages $k = 1, \ldots, H$, starting at a fixed initial state, s_0. Terminal states in finite-horizon MDPs are absorbing states. That is, if s is a terminal state and we are at time step k, every action a will map (s, k) into (s, $k + 1$) and will be reward-free i.e. $R(s, a) = 0$. Our goal is to produce a policy, π, that maps any given state into an action, such that it maximises the expected accumulated reward received for an episode of the MDP,

$$\text{argmax}_\pi E\left\{ \sum_{k=0}^{H-1} R(s_k, \pi(s_k)) \right\} \tag{1}$$

where the expectation is over $s_{k+1} \sim T(s_k, \pi(s_k), s_{k+1})$.

We assume that we have access to a simulator of the environment that given any state-action pair (s, a), where $s \in \mathcal{S}$ and $a \in A(s)$, the simulator returns the reward $R(s, a)$, a resulting state s' following the probability distribution $T(s, a, s')$ and whether s' is a terminal state. In line with previous work (6; 8; 10), we consider only *discrete* action sets and states that represent the internal state of the Atari environment such that action transitions are deterministic, that is $T(s, a, s')$ can only equal 1 for one state s' and 0 for any other state s'', $s, s', s'' \in \mathcal{S}$, $s' \neq s''$ and $a \in A(s)$. Note that while the agent can use the internal Atari game state to set the state of the simulator, it can only directly observe the Atari screen's pixel value for any given state.

2.2 Online Planning over simulators

In this paper we explore online planning over simulators by considering width-based lookahead algorithms for the Atari-2600 games. Lookaheads use a simulator of the environment to consider rewards from different action trajectories from the current state into the future. An example of this is shown in Figure 1 where a lookahead is illustrated. We define the notion of lookahead as,

Definition 1 (Lookahead)**.** A lookahead is defined as L = (N, C, s_r) where N is a set of nodes defined as state-action paths starting at the root state of the lookahead s_r, and C is a function that given a node $n \in N$ and an action $a \in A(s)$ returns the children of n, that is $n_c \in C(n, a)$ and $n_c \in N$.

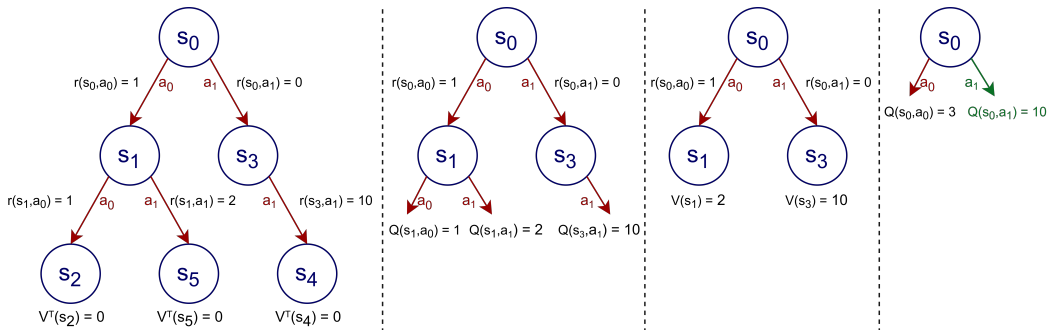

Figure 1: Example lookahead showing the action selection process for the root node s_0 following Definition 2. On the left, we have a fully built lookahead. From left to right, we show the recursive process to determine what is the action to be executed. The transition with action a_1 from s_0 shown in green in the last diagram on the right is the transition added to the lookahead's critical path (Definition 3). The V, Q, and V^T functions are as defined in Definition 2.

Through backing up the rewards for each node in the lookahead, as shown in Figure 1, an expected value can be found for each action applicable at the current state. That is, where n^s is the last state along the state-action path of node n, the operation of backing up the rewards and selecting which action to execute is,

Definition 2 (Action selection of lookahead)**.** Given a lookahead L= (N, C, s_r) (Definition 1) the action to execute a is selected at the root n_r, where $n_r^s = s_r$, by $\text{argmax}_{a \in A(n_r^s)}\{Q(n_r, a)\}$, where $Q(n_r, a) = R(n_r^s, a) + \sum_{n \in C(n_r, a)} T(n_r^s, a, n^s)V(n)$, and $V(n) = V^T(n)$, with $V^T(n)$ being a termination cost, when $\bigcup_{a \in A(n^s)} C(n, a) = \emptyset$, otherwise $V(n) = \max_a\{R(n^s, a) + \sum_{n' \in C(n,a)} T(n^s, a, n'^s)V(n')\}$.

Once an action is selected for execution, the lookahead is updated to have its root at the selected action's resulting node and the lookahead continues being constructed from the new root node. We define the action selected by the agent as a part of its *critical path*. That is,

Definition 3 (Critical Path)**.** Given the action selected a_t at each time step $t = 0, 1, \ldots, m$ following Definition 2, the critical path ρ is the sequence of states and actions $\rho = (s_0, a_0, s_1, a_1, \ldots, a_{m-1}, s_m)$, such that $s_{i+1} \sim T(s_i, a_i, \cdot)$ for $i = 0, \ldots, m-1$.

3 Related Work

3.1 MuZero

Schrittwieser et al. (11) followed up on the AlphaGo (13) and AlphaZero (14) algorithms with MuZero. AlphaZero is a planning and learning agent that achieved state-of-the-art performance on the games of Go, Chess and Shogi (15). AlphaZero learns a policy and value network that are used within a Monte Carlo Tree Search (MCTS) lookahead through sampling actions according to the policy network and evaluating the states within the lookahead with the value function. Unlike AlphaZero, MuZero does not require a simulator or model of the game environment but instead learns a model of the environment through interaction. MuZero achieved state-of-the-art performance in the Atari-2600 games when compared to existing model-free RL algorithm performances. We also explore using learnt value and policy networks within a lookahead but consider width-based methods as opposed to MCTS. MuZero's experimental setting is different to the one considered in this paper as we require access to a simulator in the planning phase and use significantly less computing power.

3.2 Width-based Planners on Atari

Here we will provide an overview of the different width-based planners which have been applied to the Atari-2600 problems. In the next Section we go into the design and implementation details that each of the following planners use.

Width-based planners (16) prioritise adding states to the lookahead with novel valuations of features that are defined over the states. *IW(1)* is a width-based *breadth-first search* that is guaranteed to run in

linear time and space as it only expands *novel* states. *IW(1)* considers a state in the lookahead as *novel* if it is the *first* state within the lookahead to make a particular feature within the feature set true. Width-based planners were first applied to the Atari-2600 games by Lipovetzky et al. (5), where they applied IW(1) over the RAM values of the game state as features. Lipovetzky et al., and subsequent works that use IW(1) (17; 18), show that it outperforms breadth-first search and UCT (19) planners.

Algorithm 1: Overview of the RIW(1) Algorithm

Input : A lookahead $L = (N, C, s_r)$, and a base policy π_b
Output Updated lookahead L
:

```
while ¬ has_solved_label(s_r) do
    s ← s_r  // complete depth-first rollout from the root node's state
    while is_novel(s) ∧ ¬ is_terminal(s) do
        s', a ← sample_unsolved_child(s, π_b)
        L ← update_lookahead(L, s, a, s'), s ← s'
    end
    update_solved_labels(s)
end
```

Bandres et al. (6) introduced a *depth-first* version of the IW(1) planner, named Rollout-IW(1) (RIW). RIW(1) aims to contain the same nodes that are expanded by the IW(1) planner. As RIW(1) performs depth-first search it was argued by Bandres et al. that it has better any-time performance than IW(1). The hypothesis for the better any-time performance of RIW(1) is that its search visits states that are further away from the initial state earlier in the search than its breadth-first search counter-part IW(1). Algorithm 1 provides an overview of RIW(1) using the base policy π_b. RIW(1) was originally defined to use a random uniform base policy π_b, however any given policy can be used instead. The function `sample_unsolved_child`, samples an action $a \sim \pi_b(s)$ and a transition $s' \sim T(s, a, \cdot)$ provided that s' has not been marked as solved. If the selected transition does not already exist in the lookahead `update_lookahead` adds it. The `update_solved_labels` function adds a solved label to the given state and back-propagates the solved label to its parents if the parent's children have all been marked as solved. Bandres et al. results showed that RIW(1) outperformed IW(1) greatly when almost real-time budgets for planning were applied.

Junyent et al. (8) follow up on Bandres et al.'s (6) use of a random policy for the base policy (π_b) of RIW with π-IW(1), an algorithm that replaces π_b with a trained policy defined over a NN. The intended effect is to orient the lookahead to promising areas of the state space. The NN from the trained policy is also used to extract features from the screen pixels for the computation of state novelty. Recently Junyent et al. (10) introduced π-IW(1)+ and π-HIW(n, 1) as follow ups of π-IW(1). π-IW(1)+ modifies π-IW(1)'s random breaking of ties for the action selection (Definition 2) to select the action with the branch of the lookahead that contains the most nodes. π-IW(1)+ also adds a learnt value function, $\tilde{V}$, which is used in the action selection (Definition 2) by modifying $V(s)$ to be $\max\{\tilde{V}(s), V^*(s)\}$, where $V^*(s)$ is $V(s)$ as described in Definition 2. π-HIW(n, 1) is a hierarchical algorithm that has a high-level planner which uses a coarse down-sampling of the screen pixels as a feature set and a low-level planner which uses π-IW(1)+ with the feature set defined through the policy network as previously described. The high-level planner uses a modified stochastic exploration policy, that selects actions with probability inversely proportional to state visitation counts.

4 Width-based Planning and Learning for the Atari Games

In this Section we step through different design considerations when constructing a width-based planning and learning algorithm. We compare the design decisions made by previous works and propose new algorithms to test over the Atari-2600 games.

4.1 Novelty Definitions: Classic and Depth-based

The novelty definition of a width-based lookahead dictates which states to prune. In Algorithm 1 the novelty definition determines the output of the `is_novel` function. We refer to IW(1)'s novelty definition as the "Classic" definition and define it as,

Definition 4 ("Classic" Novelty)**.** Given a feature set $F = \{f_1, \ldots, f_i, \ldots, f_k\}$ s.t. $f_i : S \rightarrow \{\top, \bot\}$, and a lookahead L= (N, C, s_r), a node n is novel, if n contains the first state generated s.t. $f(n^s) = \top$ for some $f \in F$, that is, $\forall n' \in N$, $f(n'^s) = \bot$ and $n' \neq n$.

The "Depth" novelty definition introduced for RIW(1) by Bandres et al. (6) is,

Definition 5 ("Depth" Novelty)**.** Given a lookahead L= (N, C, s_r), a newly generated node, $n \notin N$, reached after doing $d(n)$ actions from s_r, is novel, if $f(n^s) = \top$ for some $f \in F$, and $\forall n' \in N$, such that $d(n') \leq d(n)$, $f(n'^s) = \bot$.

Later we show that in a width-based planning and learning algorithm based upon the RIW(1) algorithm the original "Classic" novelty is competitive and can sometimes outperform the depth-based one over the Atari-2600 games. In what follows, we refer to Bandres et al.'s original configuration of RIW(1) as RIW_D and refer to RIW(1) where one replaces the "Depth" novelty definition with the "Classic" one as RIW_C.

4.2 Features for Novelty from Graphical Game Outputs

Width-based methods require a feature set F to be defined over the observable state-space S. There are two types of observations that can be used for the Atari-2600 games, the internal states of the Atari-2600 machine (the RAM), and the colours of screen pixels. Either of these enable features to be defined, as arbitrary Boolean functions over the observable variables. For the internal state observables we have b $\times$ x variables, where b is the size of the Atari memory word (8 bits) and x is the size of the physical RAM given by the number of distinct memory addresses (128 addresses). There are c $\times$ w $\times$ h screen observable variables, where c is 128, w is 160, and h is 210, corresponding to the colour depth, and the number of screen pixels along the horizontal and vertical directions.

When RIW(1) was introduced (6), Bandres et al. stated that features capturing "meaningful structure" would yield better results than using raw features. Hence, Bandres et al. mapped the observable screen variables into the feature set B-PROST, first proposed by Liang et al. (3). The B-PROST feature set attempts to capture temporal and spatial relationships between the past and present screen pixel values. In order to compute the set of B-PROST features there are a number of steps required. First a set of basic features needs to be computed through dividing the screen into 16×14 tiles comprised of 10×15 pixels. For each tile, (w, h), where $w \in \{1, \ldots, 16\}$ and $h \in \{1, \ldots, 14\}$, there are K features where K is equal to the colour depth of the Atari-2600 pixels (128). The basic feature in the B-PROST set is $f_{w,h,c}$, where $c \in \{1, \ldots, K\}$ is true if the tile (w, h) contains at least one pixel with the colour value c. A second tier of features, the Basic Pairwise Relative Offsets in Space (B-PROS) set, is computed from the basic ones. A B-PROS feature $f_{c_1,c_2,i,j}$, is true if $f_{w,h,c_1} \wedge f_{w+i,h+j,c_2}$ for any w, h. Finally, a third tier of features, the Basic Pairwise Relative Offsets in Time (B-PROT) set, are computed. A B-PROT feature considers the current screen's tiles (w, h) and the previous game screen's tile (w', h') so that a feature $f^t_{c_1,c_2,i,j}$ is true if $f_{w,h,c_1} \wedge f_{w'+i,h'+j,c_2}$ for any w, h where $w' = w$ and $h' = h$. The B-PROST set is the union of basic, B-PROS, and B-PROT feature sets.

The feature set can also be defined dynamically through a NN (8). π-IW, π-IW(1)+ and the lower level planner of π-HIW(n, 1) use a feature set that is defined as the output values of the rectified linear units from the last hidden layer of the policy NN treating zero values as $\bot$ and positive as $\top$. The policy NN input are the last four screens, processed to map colours to a suitably defined grayscale, and down sampled to a size of 84×84. While the policy network is being trained the features extracted through it will also change. This is similar to Dittadi et al. (9), who use Variational Autoencoders (VAE) to learn a set of features from the Atari game screen using a training set of game screens created from a RIW(1) execution using B-PROST. RIW(1) using the VAE features was shown to outperform RIW(1) using the B-PROST features.

While width-based planning methods using both the BPROST and NN extracted feature sets have been shown to perform well over the Atari games, previous width-based methods have not tested simpler feature sets defined directly over the screen pixel values. With the motivation of presenting a simpler width-based algorithm, we define our feature set directly over the values of the current down sampled 84×84, 8-bit grayscaled, observable screen variables. Each feature is defined as $f_{i,j,c}$ and is true if the downsampled pixel (i, j) has the grayscaled colour c, where $i, j \in \{1, \ldots, 84\}$ and $c \in \{1, \ldots, 256\}$. Despite using a simpler feature set than previous work, in the next Section we show that our algorithm outperforms the methods that use dynamically defined NN based features.

Algorithm 2: Novelty guided Critical Path Learning (N-CPL)

```
// Perform K training iterations
for i = 0, ..., K do
    T^i ← ∅, E^i ← ∅ // Iteration's critical path transitions and episode rewards
    while ¬ train_interval_exhausted() do
        s ← s_0, R ← 0, L ← initialise_lookahead(s_0)
        while ¬ is_terminal(s) do
            L ← RIW(L, π_b) // Algorithm 1
            a, r, s', L ← select_next_transition(L, V^t) // Selected using Def.2
            T^i ← T^i ∪ (s, a, r, s'), s ← s', R ← R + r
        end
        E^i ← E^i ∪ R // Episode rewards from current iteration
    end
    // Train and update network parameters according to learning schedule
    π_b, V^t ← update_network_parameters(T^i, E^i)
end
```

4.3 Learning Base Policies and Termination Costs

AlphaGo (13) and π-IW (8) showed the power of using a learnt base policy defined through a NN in order to guide a lookahead search. Similarly, AlphaGo and π-IW(1)+ (10) also use a learnt value function defined through a NN. π-IW(1)+ used its learnt value function to modify the definition of $V(n)$ (Definition 2) allowing the rewards received from the transitions in the lookahead to sometimes be ignored in preference of the value network's valuation.

We propose a new algorithm based on Algorithm 1, Novelty guided Critical Path Learning or N-CPL for short, that incorporates both a learnt policy and value function network. An outline of N-CPL is shown in Algorithm 2. Like π-IW does, N-CPL defines the base policy used by Algorithm 1 to be a policy network. Besides that, N-CPL uses a cost-to-go approximation which we implement with a NN as a value function, which is evaluated at the non-terminal leaf nodes of the lookahead. Using cost-go-approximations has been shown to significantly improve the performance of width-based lookaheads over stochastic shortest paths (20), but rather than using simulations to obtain the cost-to-go estimates, we rely on a learnt heuristic function. That is, instead of assigning termination costs, V^T, of 0 as done by previous width-based methods, if the state is not terminal the valuation of the learnt value function network is used. We note that unlike π-IW(1)+ we do not use the learnt value function to modify the $V(s)$ definition as defined in Definition 2.

4.4 Learning from Critical Paths

The policy network of N-CPL uses the state action pairs, (s_j, a_j) for $j = 0, \ldots, H-1$, of previous episodes performed by N-CPL with NN parameters, $<\theta_i^\pi, \theta_i^V>$, in order to train new NN parameters θ_{i+1}^π. This is similar to how π-IW(1) trains its policy function, except for the fact that π-IW(1) uses the Q values within the lookahead tree. If multiple actions in π-IW's lookahead have the same Q value for a given state, instead of the training vector assigning a probability of one to the executed action, π-IW uniformly distributes the probability across the actions with the same Q values. We have taken the simpler approach of just using 1-hot encodings for the single selected action along the critical path (Definition 3) of the N-CPL algorithm. Curating the training dataset in this way also means N-CPL does not need access to the internal data structures of the planning agent itself but instead can externally observe any agent interacting with the environment in order to acquire the training data.

Influential deep RL algorithms such as DQN (2) which have been applied to the Atari-2600 games rely on evaluating an ϵ-greedy policy defined over the parameters of its network in order to sample transitions and use Q-learning updates on the parameters of the network. We follow this strategy and perform Temporal Differential (TD) (21) learning to train the value network. The selection of the transitions that are used for the TD learning determines what the value function is estimating. That is, TD's task is to estimate the expected accumulated rewards from the given state following the policy which underlies the transitions that it is trained on. In N-CPL the transitions within the

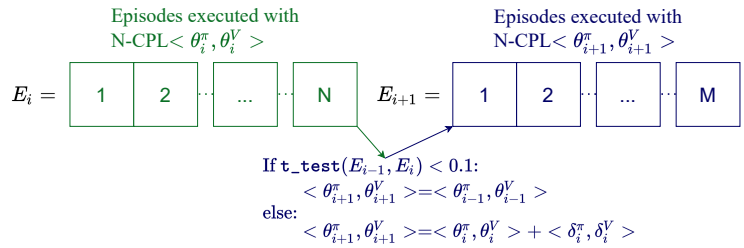

Figure 2: Illustration of the schedule for the network parameter updates, $<\theta^\pi, \theta^V>$, of N-CPL. The E arrays contain episodes executed by N-CPL and each episode in the array is represented by a square in the diagram. The `t_test` function returns the p-value for Welch's t-test (22) of the episode rewards executed with the old $i-1$ parameters being better than the new ones. δ_i^π and δ_i^V are real vectors of the same dimensions as θ_i^π and θ_i^V respectively. δ_i^π and δ_i^V are functions over E_i.

lookahead follow the base policy, π_b which is being learnt by aiming to mimic the policy induced from the N-CPL lookahead, $\pi_{\text{N-CPL}}$. The N-CPL lookahead through selecting actions according to Definition 2 can be seen as a policy improvement operator over π_b and hence the execution of $\pi_{\text{N-CPL}}$ is not necessarily equivalent to the π_b. Therefore it does not make sense to approximate the expected accumulated reward of the lookahead with the expected accumulated reward of π_b. Instead, as shown in Line 8 of Algorithm 2, N-CPL trains only the transitions on the critical path of the lookahead (Definition 3), that is, the transitions selected by $\pi_{\text{N-CPL}}$. This results in the value function approximating the expected accumulated reward of executing $\pi_{\text{N-CPL}}$ from a given state.

4.5 Adding a Learning schedule

The previous width-based planning and learning methods continuously learn and update their policy and value networks, while a key mechanism of the Alpha-Zero algorithm (14) is the use of a learning schedule. Alpha-Zero evaluates each new set of network parameters θ' that are trained against the current set of network parameters θ to ensure θ' improves AlphaZero's performance. Here we introduce a general learning schedule mechanism that is applicable to sequentially executed and trained planning and learning methods applied to single-player domains, that N-CPL uses for updating its network parameters for its policy network, θ^π and value function network, θ^V. As illustrated in Figure 2, the learning schedule determines whether network parameter updates $<\theta_i^\pi, \theta_i^V>$ can be accepted or if the $<\theta_{i-1}^\pi, \theta_{i-1}^V>$ parameters are kept, by evaluating their performance when used within N-CPL. This test is implemented in the `update_network_parameters` function shown in Algorithm 2. For the test N-CPL performs a Welch's t-test (22) on its performance with the new i parameters vs. the old $i-1$ parameters. The update is rejected if the t-test suggests, with a p-value of less than 0.1, that performance could deteriorate if the new parameters were accepted. This training and parameter update schedule allows learning steps to be completed at each time step like done by π-IW (8), except that the updated parameters are not used for data generation by N-CPL until they have been accepted by the proposed learning schedule.

5 Experimental Study

We benchmark width-based planning and learning methods with variations of the design decisions explained in the previous section. Here we explain our experimental methodology and provide results across the different algorithms over the Atari-2600 games.

5.1 Methodology

Given vast compute resources it would be preferable to conduct a full ablation study over the Atari-2600 benchmark on each element of a width-based planning and learning algorithm discussed in the previous Section. However, due to computational constraints we instead select five planners which will provide the most insight. Two of the planners we evaluate are based on RIW(1) without learnt policy or value function networks. One of the planners uses the "Depth" definition of novelty (Definition 5) and the other uses the "Classic" definition (Definition 4), we name these version RIW_D

and RIW_C respectively. Note that RIW_D is as described in Bandres et al.(6), except that the features are defined directly over the screen's pixel values as discussed in the previous Section. The other 2 width-based planners we benchmark are two versions of N-CPL, as previously described. Again we test both the "Classic" and "Depth" novelty definitions (Definitions 4, 5), and refer to them as N-CPL and N-CPL_D respectively. For the policy networks of N-CPL and N-CPL_D we use the same architecture used by Mnih et al. (2). The value network uses the same architecture as the policy network except that instead of the output layer being a dense softmax layer with an output for each action, the output layer of the value network is a dense linear layer with a single output value. Additionally we test a version of N-CPL that does not prune for novelty, we refer to as CPL, i.e. for CPL the `is_novel` function in Algorithm 1 always returns true.

We compare RIW_D, RIW_C, CPL, N-CPL, and N-CPL_D to the π-IW, π-IW(1)+ and π-HIW(n, 1) planners with the results as given by Junyent et al. (10). We do not directly compare our results to those given in the original RIW(1) planner as Bandres et al. (6) use a different experimental design. That is, Bandres et al. and previous works such as Lipovetzky et al. (5) benchmarked their width-based planners over the Atari-2600 games all using the full action set of 18 actions per state. We found in the code provided for the π-IW(1) work that it was benchmarked against the games using the minimal action set for each game. Using the minimal action set results in many games having much smaller branching factors, for example, instead of Breakout having a branching factor of 18, it has a branching factor of just 4. Additionally it is worth noting that the true average branching factor of each game is often much smaller than the minimal action set (23) and learning the minimal action set can help avoid unnecessary simulator interactions (18). Due to computational constraints we could not benchmark our algorithms over the games with full, minimal and learnt minimal actions sets. Instead, we decided to benchmark using the minimal action set as Junyent et al. (10) do.

The results are also not directly compared with those from MuZero due to discrepancies in the evaluation protocols and computing resource requirements. For example, MuZero uses a smaller frameskip for the environment time steps and uses a longer allowed episode length of 108,000 frames compared to the 18,000 frame maximum episode length we impose. While our experiments run on a single vCPU for each trial for both training and evaluation, MuZero required 40 third generation Google Cloud TPUs for each run, 8 for training and 32 for its self-play. Furthermore the results for MuZero on each domain were only made available for a 20 billion frame training budget. We do however provide comparisons with the RL algorithms DQN (2) and Rainbow (4), along with all the necessary caveats about differences in evaluation protocols, in the Supplementary Material.

For the evaluation of each algorithm on each game we run 5 independent trials. Once training has completed we evaluate each trial over 10 episodes. Following previous width-based planning papers (6; 8; 10) we use a frameskip of 15. We keep our experimental settings the same as Junyent et al. (10) including a training budget of 2×10^7 simulator interactions and allowing 100 simulator interactions at each planning time step, which allows almost real-time planning. Note that previous width-based algorithms have varied in how they apply planning budgets, Lipovetzky et al. enforce a budget of 30,000 simulator interactions with a frameskip of 5, while Bandres et al. enforce time budgets of 0.5 and 32 seconds with a frameskip of 15. We ran 80 independent trials at once over 80 Intel Xeon 2.10GHz processors with 720GB of shared RAM, limiting each trial to run on a single vCPU. The average vCPU run-time per time step needed across both the planning and learning steps were 1.28 and 1.11 seconds for N-CPL, and N-CPL_D respectively, resulting in each trial taking just under 3 days to complete. For RIW_C and RIW_D, which do not require any learning steps or evaluation of NNs, the average run-times per step were 0.55 and 0.54 seconds respectively. Note that given Atari operates at 60 frames per second and we use a frameskip of 15 a real-time planner would be required to execute with a run-time of 0.25 seconds per time step.

The transitions within the lookahead are cached inline with previous width-based planners (5; 6; 8; 10). That is, when the search revisits a transition between two nodes of the lookahead within the same episode, the simulator does not need to be recalled and hence does not affect the simulator budget. Also following previous work, transitions that are cached from previous time steps are not considered by the novelty Definitions 4, and 5, and hence will never be pruned.

5.2 Results

Table 1 summarises the results of N-CPL, N-CPL_D, CPL, RIW_C, RIW_D, π-IW(1), π-IW(1)+ and π-HIW(n, 1). Using the pairwise comparison of the different algorithms across the 53 games it is

Table 1: A pairwise comparison of the width-based planning algorithms over the full benchmark set made up of 53 Atari Games. Numbers represent the number of games an algorithm had a higher average evaluation score over the 5 learning trials than the algorithm it is being compared to.

	Number of games with higher average score than								
	N-CPL	N-CPL$_D$	CPL	RIW$_C$	RIW$_D$	π-IW	π-IW+	π-HIW	**Total (ave. win %)**
N-CPL		26	35	49	47	32	39	32	**260 (70.1%)**
N-CPL$_D$	26		29	48	46	32	38	30	**249 (67.1%)**
CPL	18	24		43	45	31	39	27	**227 (61.2%)**
RIW$_C$	3	4	10		23	19	18	17	**94 (25.3%)**
RIW$_D$	5	6	8	29		20	18	17	**103 (27.8%)**
π-IW	20	20	22	33	32		30	25	**182 (49.1%)**
π-IW+	14	15	14	35	35	23		23	**159 (42.9%)**
π-HIW	21	23	26	36	36	28	30		**200 (53.9%)**

Table 2: Same as Table 1 but for games with a Branching Factor $\geq$ 10 (33 Games).

	Number of games with higher average score than								
	N-CPL	N-CPL$_D$	CPL	RIW$_C$	RIW$_D$	π-IW	π-IW+	π-HIW	**Total (ave. win %)**
N-CPL		14	22	30	29	23	28	22	**168 (72.7%)**
N-CPL$_D$	18		18	31	29	23	27	21	**167 (72.3%)**
CPL	11	15		26	28	23	28	21	**152 (65.8%)**
RIW$_C$	2	1	7		16	15	16	13	**70 (30.3%)**
RIW$_D$	3	3	5	16		16	16	13	**72 (31.2%)**
π-IW	9	9	10	17	16		19	14	**94 (40.7%)**
π-IW+	5	6	5	17	17	14		11	**75 (32.5%)**
π-HIW	11	12	12	20	20	19	22		**116 (50.2%)**

clear that N-CPL$_D$ and N-CPL are the most performant. Comparing the "Depth" vs. "Classic" novelty definition methods as RIW$_D$ vs. RIW$_C$, the former performs better than the latter. The superiority of the "Depth" over the "Classic" definition of novelty does not follow when using our CPL method. The "Classic" method of N-CPL slightly outperforms the "Depth" method N-CPL$_D$, with Table 1 indeed showing that N-CPL is the best performing algorithm overall. Interestingly our CPL method that does not use novelty pruning, still outperforms all previous methods which shows the large contribution learning and using the policy and value function networks, as described in the previous Section, has on performance.

To better understand the performance of the algorithms we segment the benchmark set according to a couple of different characteristics. The game characteristics we examine are the *branching factor*, and the *sparseness* of meaningful reward feedback (SMRF). We consider rewards as meaningful when they provide information to a player about how to maximise the accumulated reward of an episode. For a given game, SMRF is determined by executing a random policy and a Real-Time Dynamic Planner (RTDP) (24) over each of the games. RTDP is an online planner that uses a one step lookahead (Definition 1) and an approximation for the termination cost V^t at each of the leaf nodes. For the approximation of $V^t(s')$ we use the accumulated reward from a random policy executed from s' for 10 time steps. We run both the random policy and RTDP for 50 time steps (750 frames). If the results from the RTDP planner are not better than the random policy according to Welch's t-test (22) with $p < 0.1$, the game is classified as having SMRFs. The results of the random policy vs. RTDP can be found in the Supplementary Material. For example, in the game of Pong, RTDP will be able to discover states through its 10 step approximation of $V^t(s')$ where either player has scored. Using information from $V^t(s')$ about which players have scored, RTDP will be able to have a better informed policy than the random policy, so Pong would not be considered a SMRF game. In a game like Skiing, where a skier is required to ski down a mountain and pass through gates on its path down, RTDP will not discover any meaningful rewards in its 10 step rollouts. This is because in Skiing, despite a constant negative reward at each time step, there is no meaningful reward feedback until the skier reaches the bottom of the mountain where a negative reward is applied for each gate the skier missed.

Table 2 groups the games according to their branching factor. Comparing Table 2 and 1 we can see that for games with larger branching factors, the relative performance gap between our N-CPL$_D$ and N-CPL planners and Junyent et al.'s π-IW(1), π-IW(1)+ and π-HIW(n, 1) increases as the branching factor increases. For example N-CPL and N-CPL$_D$ perform better than π-IW(1)+ in 28/33 (84.8%) games and 27/33 (81.8%) games respectively for the games with a branching factor greater or equal to 10. However, for games with a branching factor less than 10, N-CPL and N-CPL$_D$ only perform better than π-IW(1)+ in 11/20 (55%) and 11/21 (55%) games respectively.

Table 3: Same as Table 1 but for SMRF games (12 Games).

	Number of games with higher average score than								
	N-CPL	N-CPL$_D$	CPL	RIW$_C$	RIW$_D$	π-IW	π-IW+	π-HIW	**Total (ave. win %)**
N-CPL		7	5	10	10	7	9	7	**55 (65.5%)**
N-CPL$_D$	4		3	9	9	7	9	6	**47 (56%)**
CPL	7	9		11	11	8	9	7	**62 (73.8%)**
RIW$_C$	1	2	1		6	6	3	4	**23 (27.4%)**
RIW$_D$	1	2	1	5		6	3	4	**22 (26.2%)**
π-IW	4	4	4	5	5		4	3	**29 (34.5%)**
π-IW+	3	3	3	9	9	8		5	**40 (47.6%)**
π-HIW	5	6	5	8	8	9	7		**48 (57.1%)**

Table 3 compares the pairwise performance for games classified as SMRF games. Table 3 clearly shows that the dominant method for the SMRF games is CPL, that is, the algorithm without novelty pruning. Table 3 also shows that the "Classic" novelty (Definition 4) methods outperform "Depth" novelty (Definition 5). These observations, that contradict previous claims in the literature, required careful analysis. We observed that the "Classic" method prunes states more aggressively, meaning it is more likely to reach states that are further away from the root node compared with the "Depth" definition. Similarly, as the CPL method does not prune any states due to novelty, CPL's depth-first lookahead trajectories will always reach the lookahead search horizon of 100 time steps at least once, given that the lookahead simulator budget is 100 time steps. This results in CPL on average searching for states that are further away from the root node than any of the novelty pruning methods. By definition, high rewards for SMRF games have a higher probability of being further away than games with dense rewards. Therefore, CPL and the "Classic" novelty methods are more likely to discover the meaningful rewards by searching deeper in the lookahead. Interestingly CPL and N-CPL still outperform, yet are close to π-IW(1)+ and π-HIW(n, 1) on the SMRF games, despite π-IW(1)+ and π-HIW(n, 1) being motivated by such domains.

6 Discussion

We have found significant discrepancies in the experimental settings used in the previous width-based planning papers for evaluating their algorithms over the Atari-2600 games. We believe a clear and consistent evaluation protocol should be set out for planning based algorithms applied to the Atari-2600 games to facilitate the direct comparison of their results. This could be similar to the evaluation protocol for the Atari-2600 games set out by Machado et al. (25), which was mainly focused towards RL agents and included recommendations on episode termination, setting of hyper-parameters, measuring training data, summarising learning performance and injecting stochasticity. However Machado et al. do not discuss evaluation settings that are vital to the deterministic planning setting we have explored in this paper, such as planning budgets, and caching of transitions within lookaheads. We hope that by having identified some of the discrepancies in the experimental settings of previous width-based algorithms, such as the size of the action set and the planning budget used, future research in planning agents for the Atari-2600 games can be more easily assessed. We were able to observe interesting patterns in the relative performance of algorithms through segmenting the Atari-2600 games by their different game characteristics. We are not aware of other works that analyse the performance of agents in regards to the characteristics of specific Atari-2600 games. We believe this taxonomy will provide useful insights into the behaviour of agents on the Atari-2600 games.

In this paper we have focused on width-based planning methods that have been applied over the Atari-2600 games. It is important to note though that these algorithms are defined in a general way to operate over MDPs. We proposed new width-based planning and learning algorithms through the examination of different design decisions made by previous implementations of width-based planners. These new algorithms, particularly N-CPL, are simpler implementations than the previously introduced width-based planning and learning algorithms π-IW(1)+ and π-HIW(n, 1). N-CPL defines its features directly over the grayscaled pixel colours of the game screen and uses a simplified novelty definition. Furthermore, N-CPL learns a value function which is only used for cost-to-go approximations at the leafs of the lookahead search tree. N-CPL also uses a methodical learning schedule we introduced for training both its policy and value networks. We found N-CPL to outperform π-IW(1), π-IW(1)+ and π-HIW(n, 1) not only across the Atari-2600 benchmark, but also over subsets of games with large branching factors and games with sparse meaningful rewards. These results show that N-CPL's integration of planning and learning pays off for almost real-time planning over hard problems.

7 Acknowledgements and Funding Disclosure

Funding in direct support of this work: Australian Government Research Training Program Scholarship provided by the Australian Commonwealth Government and the University of Melbourne; the Defence Science Institute, an initiative of the State Government of Victoria; and computing resources provided by the University of Melbourne through the Melbourne Research Cloud.

References

[1] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

[2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529, 2015.

[3] Yitao Liang, Marlos C. Machado, Erik Talvitie, and Michael H. Bowling. State of the art control of atari games using shallow reinforcement learning. In *Proc. of Int'l Conf. on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 485–493. ACM, 2016.

[4] Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proc. of the National Conference on Artificial Intelligence (AAAI)*, pages 3215–3222, 2018.

[5] Nir Lipovetzky, Miquel Ramirez, and Hector Geffner. Classical planning with simulators: results on the atari video games. In *Proc. of Int'l Joint Conf. in Artificial Intelligence (IJCAI)*, pages 1610–1616, 2015.

[6] Wilmer Bandres, Blai Bonet, and Hector Geffner. Planning with pixels in (almost) real time. In *Proc. of the National Conference on Artificial Intelligence (AAAI)*, volume 32, pages 6102–6109. AAAI Press, 2018.

[7] Patrik Haslum, Nir Lipovetzky, Daniele Magazzeni, and Christian Muise. An introduction to the planning domain definition language. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 13(2):1–187, 2019.

[8] Miquel Junyent, Anders Jonsson, and Vicenc Gomez. Deep policies for width–based planning. In *Proc. of the Int'l Conf. in Automated Planning and Scheduling (ICAPS)*, volume 29, pages 646–654, 2019.

[9] Andrea Dittadi, Frederik K Drachmann, and Thomas Bolander. Planning from pixels in atari with learned symbolic representations. In *Proc. of the National Conference on Artificial Intelligence (AAAI)*, volume 35, pages 4941–4949, 2021.

[10] Miquel Junyent, Vicenç Gómez, and Anders Jonsson. Hierarchical width-based planning and learning. In *Proc. of the Int'l Conf. in Automated Planning and Scheduling (ICAPS)*, volume 31, pages 519–527, 2021.

[11] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

[12] Hector Geffner and Blai Bonet. A concise introduction to models and methods for automated planning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 8(1):1–141, 2013.

[13] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, and others. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.

[14] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, and others. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354, 2017.

[15] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[16] Nir Lipovetzky and Héctor Geffner. Width and serialization of classical planning problems. In *Proc. of European Conference in Artificial Intelligence (ECAI)*, pages 540–545, 2012.

[17] Alexander Shleyfman, Alexander Tuisov, and Carmel Domshlak. Blind search for atari-like online planning revisited. In *Proc. of Int'l Joint Conf. in Artificial Intelligence (IJCAI)*, pages 3251–3257, 2016.

[18] Yuu Jinnai and Alex Fukunaga. Learning to prune dominated action sequences in online black-box planning. In *Proc. of the National Conference on Artificial Intelligence (AAAI)*, volume 31, pages 839–845, 2017.

[19] Levente Kocsis and Csaba Szepevari. Bandit based monte carlo planning. In *Proc. of European Conference in Machine Learning (ECML)*, pages 282–293, 2006.

[20] Stefan O'Toole, Miquel Ramirez, Nir Lipovetzky, and Adrian Pearce. Width-based lookaheads augmented with base policies for stochastic shortest paths. In *ICAPS Workshop on Heuristics and Domain Independent Planning (HSDIP)*, pages 37–45, 2019.

[21] Richard S Sutton. Learning to predict by the methods of temporal differences. *Machine learning*, 3(1):9–44, 1988.

[22] B. L. Welch. The generalization of 'student's' problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35, 1947.

[23] Mark J Nelson. Estimates for the branching factors of atari games. In *Proc. of the IEEE Conference on Games.*, 2021.

[24] Andy G. Barto, S. J. Bradtke, and S. P. Singh. Real–time learning and control using asynchronous dynamic programming. *Artificial Intelligence Journal*, 72:81–138, 1995.

[25] Marlos C Machado, Marc G Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.

Additional results

Table 4: Comparing averages directly over 53 Atari-2600 games, as no confidence interval or standard deviation data is provided for the results of π-IW(1), π-IW(1)+, π-HIW(n, 1). The highest average score is highlighted in green. Freeway is excluded from this table as Junyent et al. (10) do not report the results for it due to its slow simulator time. See Table 5 for the Freeway results of RIW_D, RIW_C, N-CPL_D, and N-CPL.

GAME	RIW_D	RIW_C	CPL	N-CPL_D	N-CPL	π-IW	π-IW+	π-HIW(n,1)
Alien	4,365.20	4,478.40	6,209.00	**7,640.60**	6,943.40	3,969.78	2,585.77	4,609.18
Amidar	1,014.84	897.00	1,433.68	**2,404.60**	2,118.32	950.45	374.20	1,076.17
Assault	764.80	768.00	**3,222.98**	3,185.44	3,079.92	1,574.91	922.30	2,344.28
Asterix	52,940.00	54,090.00	39,860.00	48,364.00	48,226.00	**346,409.11**	247,063.36	90,017.25
Asteroids	1,480.20	1,397.20	6,145.20	9,000.80	**9,152.80**	1,368.55	1,490.87	990.95
Atlantis	48,930.00	46,464.00	**173,786.00**	120,650.00	119,636.00	106,212.63	143,177.73	17,539.22
BankHeist	453.46	436.90	336.40	**957.12**	709.00	567.16	256.29	501.68
BattleZone	102,780.00	88,560.00	220,680.00	165,340.00	153,880.00	69,659.40	30,848.95	**309,137.79**
BeamRider	4,124.37	3,521.80	6,164.64	3,743.80	3,560.88	3,313.11	8,428.96	**11,931.41**
Berzerk	600.00	620.00	3,148.00	4,642.20	4,120.60	1,548.23	960.03	**7,417.26**
Bowling	65.38	63.10	**161.06**	101.40	103.24	26.28	78.18	50.09
Boxing	52.44	54.58	80.96	84.30	86.40	**99.88**	88.19	6.81
Breakout	64.34	53.36	197.82	**320.64**	302.04	92.07	107.64	252.88
Centipede	52,685.12	55,495.78	53,608.80	60,157.38	62,654.16	126,488.35	**141,070.19**	80,685.48
ChopperCommand	3,768.00	3,466.00	17,908.00	4,570.00	3,786.00	11,187.44	3,431.74	**70,787.12**
CrazyClimber	40,520.00	39,387.50	78,266.00	90,332.00	91,912.00	**161,192.01**	138,648.58	102,205.99
DemonAttack	8,499.88	8,449.00	10,560.00	10,829.90	10,876.00	26,881.13	**35,022.64**	16,007.64
DoubleDunk	6.72	6.00	19.76	23.76	**23.96**	4.68	-16.80	3.51
Enduro	1.90	1.34	231.44	250.88	220.28	**506.59**	63.83	44.47
FishingDerby	-67.76	-62.46	-23.94	-29.22	-7.62	**8.89**	-28.02	-53.76
Frostbite	280.00	273.80	**9,956.80**	5,255.60	6,508.00	270.00	1,636.51	7,242.60
Gopher	6,311.43	5,990.83	11,181.60	13,326.40	12,539.60	**18,025.91**	7,061.76	15,001.18
Gravitar	1,755.00	1,725.00	**2,382.00**	2,246.00	2,284.00	1,876.80	1,532.33	1,154.01
Hero	17,438.30	17,159.70	29,200.40	34,083.40	36,099.80	**36,443.73**	22,097.39	36,231.21
IceHockey	21.48	21.92	19.62	**29.58**	26.96	-9.66	-4.02	-2.36
Jamesbond	2,378.00	2,485.00	18,666.00	17,572.00	**18,790.00**	43.20	104.91	1,380.13
Kangaroo	1,556.00	1,504.00	5,332.00	**9,222.00**	9,202.00	1,847.46	2,918.98	6,861.57
Krull	2,176.58	2,127.60	5,631.22	4,900.90	4,422.88	8,343.30	**13,014.77**	4,121.81
KungFuMaster	5,668.00	5,886.00	27,288.00	**43,520.00**	42,388.00	41,609.03	24,871.94	20,680.65
MontezumaRevenge	0.00	0.00	2.00	0.00	0.00	0.00	810.49	**5,275.89**
MsPacman	15,697.56	15,729.28	11,922.22	16,150.08	**18,285.18**	14,726.33	5,916.86	4,523.47
NameThisGame	6,247.50	6,145.25	8,287.40	8,017.20	8,339.40	12,734.85	**18,167.55**	9,977.12
Phoenix	4,992.80	4,912.60	6,616.40	8,033.40	**8,777.40**	5,905.12	7,647.67	7,508.63
Pitfall	-44.86	-62.66	-0.48	-1.68	**-0.42**	-214.75	-2.46	-128.82
Pong	-4.52	-4.92	-12.28	6.18	**10.94**	-20.42	2.14	-9.70
PrivateEye	625.72	1,249.68	153.22	120.00	100.00	452.40	1,766.13	**29,548.76**
Qbert	4,499.50	4,426.50	28,182.50	31,625.50	30,618.50	32,529.60	23,337.90	**40,449.72**
RoadRunner	17,326.00	20,580.00	86,650.00	49,828.00	57,212.00	38,764.81	43,813.29	**87,953.53**
Robotank	31.03	31.33	31.04	**37.94**	36.16	15.66	9.68	10.63
Seaquest	1,609.60	1,576.20	3,523.00	2,878.60	3,922.80	**5,916.05**	559.28	867.51
Skiing	-31,013.00	-22,234.22	-20,510.82	-29,080.30	-20,041.24	-19,188.32	**-13,852.04**	-15,417.86
Solaris	3,110.00	3,085.00	**7,741.60**	3,106.40	4,704.00	3,048.78	1,832.93	3,524.69
SpaceInvaders	2,592.40	2,622.50	3,447.40	3,680.40	**4,289.40**	2,694.09	1,622.49	2,946.18
StarGunner	17,510.00	17,597.56	18,340.00	**20,700.00**	20,320.00	1,381.24	1,642.82	1,864.64
Tennis	**1.40**	-0.53	-2.80	1.24	0.00	-23.67	-8.26	-20.00
TimePilot	24,455.00	24,342.50	22,780.00	24,950.00	24,150.00	16,099.92	11,126.86	**34,610.25**
Tutankham	159.40	154.86	184.94	181.48	203.54	**216.67**	181.44	199.06
UpNDown	39,834.00	40,649.00	59,650.80	58,783.00	58,867.20	**107,757.51**	59,497.75	80,991.07
Venture	22.00	32.00	**1,732.00**	1,466.00	1,564.00	0.00	15.68	10.73
VideoPinball	136,531.50	138,748.35	148,995.14	134,489.58	139,799.22	**514,012.51**	387,308.60	184,720.01
WizardOfWor	26,956.25	27,991.11	54,988.00	43,556.00	43,436.00	**76,533.18**	30,383.68	12,027.43
YarsRevenge	59,779.48	60,940.90	133,647.42	142,568.26	135,089.68	102,183.67	64,544.51	**159,496.20**
Zaxxon	9,342.00	9,520.00	28,102.00	**33,268.00**	30,818.00	22,905.73	10,159.01	21,135.58
Total times best	**1**	**0**	**7**	**10**	**8**	**12**	**5**	**10**

Table 5: Average scores with 90% confidence intervals over the set of 54 Atari Games. Algorithm scores that are the best according to the Welch's t-test (22) using p< 0.1 are highlighted in green.

GAME	RIW$_D$	RIW$_C$	CPL	N-CPL$_D$	N-CPL
Alien	4,365.20±471.95	4,478.40±375.51	6,209.00±415.77	7,640.60±615.52	6,943.40±507.28
Amidar	1,014.84±70.76	897.00±70.28	1,433.68±123.49	**2,404.60±69.72**	2,118.32±114.28
Assault	764.80±49.88	768.00±68.03	3,222.98±98.37	3,185.44±123.18	3,079.92±122.33
Asterix	52,940.00 ±1,486.56	54,090.00 ±1,323.40	39,860.00 ±410.25	48,364.00 ±1,844.10	48,226.00 ±1,184.51
Asteroids	1,480.20±75.48	1,397.20±113.97	6,145.20±318.23	9,000.80±143.15	9,152.80±189.24
Atlantis	48,930.00 ±4,017.54	46,464.00 ±3,132.23	**173,786.00 ±2,343.28**	120,650.00 ±892.42	119,636.00 ±830.95
BankHeist	453.46±31.86	436.90±40.51	336.40±25.25	**957.12±105.73**	709.00±76.09
BattleZone	102,780.00 ±22,127.98	88,560.00 ±16,866.49	**220,680.00 ±14,274.79**	165,340.00 ±28,612.76	153,880.00 ±28,305.50
BeamRider	4,124.37±384.99	3,521.80±371.92	**6,164.64±287.17**	3,743.80±466.05	3,560.88±415.25
Berzerk	600.00±32.84	620.00±32.37	3,148.00±386.38	4,642.20±367.59	4,120.60±423.64
Bowling	65.38±1.66	63.10±2.00	**161.06±5.46**	101.40±3.22	103.24±3.32
Boxing	52.44±2.00	54.58±2.86	80.96±2.63	84.30±2.10	86.40±0.97
Breakout	64.34±17.89	53.36±10.91	197.82±33.59	320.64±8.03	302.04±18.31
Centipede	52,685.12 ±2,811.22	55,495.78 ±1,398.86	53,608.80 ±1,567.38	60,157.38 ±1,726.88	**62,654.16 ±1,107.16**
ChopperCommand	3,768.00±634.47	3,466.00±378.06	**17,908.00±1,432.95**	4,570.00±602.78	3,786.00±601.16
CrazyClimber	40,520.00 ±550.63	39,387.50 ±486.25	78,266.00 ±3,572.79	90,332.00 ±2,414.91	91,912.00 ±2,184.99
DemonAttack	8,499.88±252.38	8,449.00±320.73	10,560.00±188.97	10,829.90±274.19	10,876.00±193.47
DoubleDunk	6.72±0.84	6.00±0.94	19.76±1.02	23.76±0.15	**23.96±0.07**
Enduro	1.90±0.57	1.34±0.38	231.44±10.04	**250.88±8.23**	220.28±9.71
FishingDerby	-67.76±1.96	-62.46±2.20	-23.94±4.32	-29.22±4.30	**-7.62±5.99**
Freeway	5.50 ± 0.24	5.52 ± 0.28	28.86±0.45	29.02 ± 0.45	28.96 ± 0.30
Frostbite	280.00±4.56	273.80±3.72	**9,956.80±1,066.62**	5,255.60±389.33	6,508.00±588.79
Gopher	6,311.43±440.94	5,990.83±479.70	11,181.60±72.70	**13,326.40±240.79**	12,539.60±158.58
Gravitar	1,755.00±171.35	1,725.00±192.75	2,382.00±228.46	2,246.00±201.67	2,284.00±231.37
Hero	17,438.30 ±1,011.92	17,159.70 ±949.20	29,200.40 ±1,804.42	34,083.40 ±938.86	**36,099.80 ±553.32**
IceHockey	21.48±0.73	21.92±0.83	19.62±1.09	**29.58±0.90**	26.96±1.03
Jamesbond	2,378.00±1,224.25	2,485.00±1,210.76	18,666.00±598.37	17,572.00±820.54	18,790.00±907.58
Kangaroo	1,556.00±203.70	1,504.00±151.18	5,332.00±727.16	9,222.00±546.79	9,202.00±572.19
Krull	2,176.58±89.61	2,127.60±88.85	**5,631.22±272.50**	4,900.90±141.45	4,422.88±149.38
KungFuMaster	5,668.00 ±425.75	5,886.00 ±521.52	27,288.00 ±1,908.35	43,520.00 ±1,108.67	42,388.00 ±643.13
MontezumaRevenge	0.00±0.00	0.00±0.00	2.00±3.26	0.00±0.00	0.00±0.00
MsPacman	15,697.56 ±1,148.98	15,729.28 ±1,138.48	11,922.22 ±1,044.99	16,150.08 ±1,064.72	**18,285.18 ±703.36**
NameThisGame	6,247.50±98.52	6,145.25±91.49	8,287.40±117.61	8,017.20±129.51	8,339.40±109.81
Phoenix	4,992.80±242.42	4,912.60±262.97	6,616.40±316.44	8,033.40±687.80	8,777.40±776.53
Pitfall	-44.86±22.53	-62.66±26.84	-0.48±0.78	-1.68±1.33	-0.42±0.52
Pong	-4.52±1.49	-4.92±1.28	-12.28±1.29	6.18±1.29	**10.94±1.22**
PrivateEye	625.72±685.72	1,249.68±938.31	153.22±22.74	120.00±9.30	100.00±0.00
Qbert	4,499.50±756.07	4,426.50±688.09	28,182.50±2,855.33	**31,625.50±406.22**	30,618.50±693.66
RoadRunner	17,326.00 ±3,626.95	20,580.00 ±3,760.02	**86,650.00 ±3,065.43**	49,828.00 ±3,244.08	57,212.00 ±4,094.90
Robotank	31.03±1.16	31.33±1.33	31.04±0.63	**37.94±0.73**	36.16±0.84
Seaquest	1,609.60±291.85	1,576.20±281.51	3,523.00±255.00	2,878.60±285.43	**3,922.80±245.47**
Skiing	-31,013.00 ±700.82	-22,234.22 ±872.31	-20,510.82 ±1,199.55	-29,080.30 ±1,034.25	-20,041.24 ±1,139.52
Solaris	3,110.00±232.31	3,085.00±532.56	**7,741.60±825.51**	3,106.40±169.24	4,704.00±615.29
SpaceInvaders	2,592.40±301.45	2,622.50±303.10	3,447.40±269.92	3,680.40±310.85	**4,289.40±301.15**
StarGunner	17,510.00±240.75	17,597.56±397.74	18,340.00±217.12	20,700.00±178.37	20,320.00±335.94
Tennis	1.40±1.53	-0.53±1.52	-2.80±0.93	1.24±1.45	0.00±1.51
TimePilot	24,455.00±615.53	24,342.50±544.82	22,780.00±839.59	24,950.00±528.00	24,150.00±670.57
Tutankham	159.40±4.77	154.86±4.86	184.94±4.59	181.48±3.80	**203.54±4.82**
UpNDown	39,834.00±830.87	40,649.00±822.57	59,650.80±1,078.03	58,783.00±604.10	58,867.20±752.05
Venture	22.00±20.96	32.00±23.00	**1,732.00±49.99**	1,466.00±176.15	1,564.00±149.66
VideoPinball	136,531.50 ±8,459.01	138,748.35 ±8,349.10	148,995.14 ±8,492.72	134,489.58 ±7,483.99	139,799.22 ±7,019.84
WizardOfWor	26,956.25 ±2,793.49	27,991.11 ±2,907.59	**54,988.00 ±1,906.42**	43,556.00 ±4,275.90	43,436.00 ±4,142.91
YarsRevenge	59,779.48 ±2,070.84	60,940.90 ±2,234.00	133,647.42 ±4,698.49	**142,568.26 ±3,934.50**	135,089.68 ±4,813.68
Zaxxon	9,342.00 ±944.44	9,520.00 ±1,221.31	28,102.00 ±1,264.90	**33,268.00 ±1,192.32**	30,818.00 ±1,722.99
Best algorithm (t-test, p <0.1)	0	0	11	9	9

Table 6: Comparison of experimental settings used for the results of the different algorithms. Note that the train budget includes all the simulator interactions used by the lookahead algorithms even though only a small fraction of simulator interactions are used for training the networks directly.

Algorithm	**Frameskip**	**Max. ep. length (Frames)**	**Train Budget (Sim. Interactions)**	**Lookahead Budget (Sim. Interactions)**	**Starts**	**Loss of Life signal**
RIW$_D$, RIW$_C$	15	18,000	0	100	-	No
N-CPL$_D$, N-CPL, CPL, π-IW, π-IW+, π-HIW(n,1)	15	18,000	20×10^6	100	-	No
DQN	4	18,000	50×10^6	NA	Rand.no-ops	Yes
Rainbow	4	108,000	50×10^6	NA	Human and Rand. no-ops	Yes

Table 7: Comparison of N-CPL with a Human player's scores and the model-free RL algorithm DQN scores as reported by Mnih et al.(2). Note that the experimental settings are different between N-CPL and DQN(2), in terms of training budget, frame skips, using no-op starts and loss of life signal, see Table 6 for a comparison of experimental settings.

GAME	**Human**	**DQN**	**RIW_C+CPV**
Alien	6,875.00	3,069.00	**6,943.40**
Amidar	1,676.00	739.50	**2,118.32**
Assault	1,496.00	**3,359.00**	3,079.92
Asterix	8,503.00	6,012.00	**48,226.00**
Asteroids	**13,157.00**	1,629.00	9,152.80
Atlantis	29,028.00	85,641.00	**119,636.00**
BankHeist	**734.40**	429.70	709.00
BattleZone	37,800.00	26,300.00	**153,880.00**
BeamRider	5,775.00	**6,846.00**	3,560.88
Bowling	**154.80**	42.40	103.24
Boxing	4.30	71.80	**86.40**
Breakout	31.80	**401.20**	302.04
Centipede	11,963.00	8,309.00	**62,654.16**
ChopperCommand	**9,882.00**	6,687.00	3,786.00
CrazyClimber	35,411.00	**114,103.00**	91,912.00
DemonAttack	3,401.00	9,711.00	**10,876.00**
DoubleDunk	-15.50	-18.10	**23.96**
Enduro	**309.60**	301.80	220.28
FishingDerby	**5.50**	-0.80	-7.62
Freeway	29.60	**30.30**	28.96
Frostbite	4,335.00	328.30	**6,508.00**
Gopher	2,321.00	8,520.00	**12,539.60**
Gravitar	**2,672.00**	306.70	2,284.00
Hero	25,763.00	19,950.00	**36,099.80**
IceHockey	0.90	-1.60	**26.96**
Jamesbond	406.70	576.70	**18,790.00**
Kangaroo	3,035.00	6,740.00	**9,202.00**
Krull	2,395.00	3,805.00	**4,422.88**
KungFuMaster	22,736.00	23,270.00	**42,388.00**
MontezumaRevenge	**4,367.00**	0.00	0.00
MsPacman	15,693.00	2,311.00	**18,285.18**
NameThisGame	4,076.00	7,257.00	**8,339.40**
Pong	9.30	**18.90**	10.94
PrivateEye	**69,571.00**	1,788.00	100.00
Qbert	13,455.00	10,596.00	**30,618.50**
Riverraid	13,513.00	8,316.00	**22,111.20**
RoadRunner	7,845.00	18,257.00	**57,212.00**
Robotank	11.90	**51.60**	36.16
Seaquest	**20,182.00**	5,286.00	3,922.80
SpaceInvaders	1,652.00	1,976.00	**4,289.40**
StarGunner	10,250.00	**57,997.00**	20,320.00
Tennis	-8.90	-2.50	**0.00**
TimePilot	5,925.00	5,947.00	**24,150.00**
Tutankham	167.60	186.70	**203.54**
UpNDown	9,082.00	8,456.00	**58,867.20**
Venture	1,188.00	380.00	**1,564.00**
VideoPinball	17,298.00	42,684.00	**139,799.22**
WizardOfWor	4,757.00	3,393.00	**43,436.00**
Zaxxon	9,173.00	4,977.00	**30,818.00**
# Games >human		23 (47%)	**37 (76%)**
#Games >75% human		27 (55%)	**40 (82%)**
# Games Best	10 (20%)	8 (16%)	**31 (63%)**

Table 8: Comparison of N-CPL with a Human player's scores and the model-free RL algorithm Rainbow using human starts. Note that the experimental settings are different between N-CPL and Rainbow(4), in terms of the training budget, maximum episode length, frame skips, using human starts and loss of life signal, see Table 6 for a comparison of experimental settings.

GAME	Human	Rainbow	N-CPL
Alien	6875	6022.90	**6943.40**
Amidar	1676	202.80	**2118.32**
Assault	1496	**14491.70**	3079.92
Asterix	8503	**280114.00**	48226.00
Asteroids	**13157**	2249.40	9152.80
Atlantis	29028	**814684.00**	119636.00
BankHeist	734.4	**826.00**	709.00
BattleZone	37800	52040.00	**153880.00**
BeamRider	5775	**21768.50**	3560.88
Bowling	**154.8**	39.40	103.24
Boxing	4.3	54.90	**86.40**
Breakout	31.8	**379.50**	302.04
Centipede	11963	7160.90	**62654.16**
ChopperCommand	9882	**10916.00**	3786.00
CrazyClimber	35411	**143962.00**	91912.00
DemonAttack	3401	**109670.70**	10876.00
DoubleDunk	-15.5	-0.60	**23.96**
Enduro	309.6	**2061.10**	220.28
FishingDerby	5.5	**22.60**	-7.62
Freeway	**29.6**	29.10	28.96
Frostbite	4335	4141.10	**6508.00**
Gopher	2321	**72595.70**	12539.60
Gravitar	**2672**	567.50	2284.00
Hero	25763	**50496.80**	36099.80
IceHockey	0.9	-0.70	**26.96**
Kangaroo	3035	**10841.00**	9202.00
Krull	2395	**6715.50**	4422.88
KungFuMaster	22736	28999.80	**42388.00**
MontezumaRevenge	**4367**	154.00	0.00
MsPacman	15693	2570.20	**18285.18**
NameThisGame	4076	**11686.50**	8339.40
Pong	9.3	**19.00**	10.94
PrivateEye	**69571**	1704.40	100.00
Qbert	16455	18397.60	**30618.50**
RoadRunner	7845	54261.00	**57212.00**
Robotank	11.9	**55.20**	36.16
Seaquest	**20182**	19176.00	3922.80
SpaceInvaders	1652	**12629.00**	4289.40
StarGunner	10250	**123853.00**	20320.00
Tennis	-8.9	-2.20	**0.00**
TimePilot	5925	11190.50	**24150.00**
Tutankham	167.6	126.90	**203.54**
Venture	1188	45.00	**1564.00**
VideoPinball	17298	**506817.20**	139799.22
WizardOfWor	4757	14631.50	**43436.00**
Zaxxon	9173	19658.00	**30818.00**
# Games >human		31 (67%)	**34 (74%)**
#Games >75% human		36 (78%)	**37 (80%)**
# Games Best	7 (15%)	**21(46%)**	18 (39%)

Table 9: Same as Table 8 but with Rainbow using random no-op starts.

GAME	Human	Rainbow	N-CPL
Alien	6875	**9491.70**	6943.40
Amidar	1676	**5131.20**	2118.32
Assault	1496	**14198.50**	3079.92
Asterix	8503	**428200.30**	48226.00
Asteroids	**13157**	2712.80	9152.80
Atlantis	29028	**826659.50**	119636.00
BankHeist	734.4	**1358.00**	709.00
BattleZone	37800	62010.00	**153880.00**
BeamRider	5775	**16850.20**	3560.88
Bowling	**154.8**	30.00	103.24
Boxing	4.3	**99.60**	86.40
Breakout	31.8	**417.50**	302.04
Centipede	11963	8167.30	**62654.16**
ChopperCommand	9882	**16654.00**	3786.00
CrazyClimber	35411	**168788.50**	91912.00
DemonAttack	3401	**111185.20**	10876.00
DoubleDunk	-15.5	-0.30	**23.96**
Enduro	309.6	**2125.90**	220.28
FishingDerby	5.5	**31.30**	-7.62
Freeway	29.6	**34.00**	28.96
Frostbite	4335	**9590.50**	6508.00
Gopher	2321	**70354.60**	12539.60
Gravitar	**2672**	1419.30	2284.00
Hero	25763	**55887.40**	36099.80
IceHockey	0.9	1.10	**26.96**
Kangaroo	3035	**14637.50**	9202.00
Krull	2395	**8741.50**	4422.88
KungFuMaster	22736	**52181.00**	42388.00
MontezumaRevenge	**4367**	384.00	0.00
MsPacman	15693	5380.40	**18285.18**
NameThisGame	4076	**13136.00**	8339.40
Pong	9.3	**20.90**	10.94
PrivateEye	**69571**	4234.00	100.00
Qbert	16455	**33817.50**	30618.50
RoadRunner	7845	**62041.00**	57212.00
Robotank	11.9	**61.40**	36.16
Seaquest	**20182**	15898.90	3922.80
SpaceInvaders	1652	**18789.00**	4289.40
StarGunner	10250	**127029.00**	20320.00
Tennis	-8.9	**0.00**	**0.00**
TimePilot	5925	12926.00	**24150.00**
Tutankham	167.6	**241.00**	203.54
Venture	1188	5.50	**1564.00**
VideoPinball	17298	**533936.50**	139799.22
WizardOfWor	4757	17862.50	**43436.00**
Zaxxon	9173	22209.50	**30818.00**
# Games >human		**37 (80%)**	34 (74%)
#Games >75% human		**38 (83%)**	37 (80%)
# Games Best	6 (13%)	**31(67%)**	10(22%)

Table 10: Mean with standard deviations of 20 RTDP vs Random episodes for the SMRF test. Bold domains are classified as having SMRF.

GAME	RTDP	Random
Alien	379.50 ± 180.1	135.00 ± 39.4
Amidar	30.90 ± 18.0	8.70 ± 15.8
Assault	139.65 ± 38.3	49.35 ± 27.6
Asterix	625.00 ± 91.5	155.00 ± 70.5
Asteroids	397.50 ± 137.6	74.00 ± 42.0
Atlantis	1350.00 ± 915.7	635.00 ± 979.9
BankHeist	27.50 ± 20.7	3.50 ± 4.8
BattleZone	3350.00 ± 2725.3	450.00 ± 669.0
BeamRider	121.00 ± 30.7	50.60 ± 46.7
Berzerk	324.00 ± 109.4	102.50 ± 68.0
Bowling	0.00 ± 0.0	0.00 ± 0.0
Boxing	12.90 ± 5.8	−2.00 ± 4.1
Breakout	3.10 ± 0.8	1.00 ± 0.7
Centipede	2189.75 ± 226.5	548.85 ± 565.0
ChopperCommand	790.00 ± 260.6	205.00 ± 120.3
CrazyClimber	715.00 ± 127.6	395.00 ± 124.4
DemonAttack	68.50 ± 9.6	29.50 ± 12.4
DoubleDunk	−0.20 ± 1.4	−1.90 ± 1.3
Enduro	0.15 ± 0.5	0.05 ± 0.2
FishingDerby	−1.70 ± 3.1	−9.70 ± 3.0
Freeway	0.00 ± 0.0	0.00 ± 0.0
Frostbite	122.00 ± 21.1	23.50 ± 16.5
Gopher	0.00 ± 0.0	0.00 ± 0.0
Gravitar	177.50 ± 195.2	0.00 ± 0.0
Hero	2206.50 ± 886.9	36.25 ± 41.4
IceHockey	0.80 ± 0.8	−0.40 ± 0.8
Jamesbond	15.00 ± 22.9	5.00 ± 15.0
Kangaroo	100.00 ± 100.0	10.00 ± 43.6
Krull	104.50 ± 38.4	19.50 ± 18.8
KungFuMaster	130.00 ± 145.3	5.00 ± 21.8
MontezumaRevenge	0.00 ± 0.0	0.00 ± 0.0
MsPacman	396.50 ± 193.9	213.00 ± 163.7
NameThisGame	138.00 ± 41.1	44.50 ± 35.3
Phoenix	346.00 ± 93.0	110.00 ± 66.8
Pitfall	0.00 ± 0.0	−1.10 ± 4.8
Pong	−0.65 ± 1.4	−3.30 ± 1.0
PrivateEye	0.00 ± 0.0	4.75 ± 21.9
Qbert	606.25 ± 50.5	77.50 ± 125.5
Riverraid	862.50 ± 210.2	321.50 ± 70.0
RoadRunner	15.00 ± 65.4	5.00 ± 21.8
Robotank	1.05 ± 0.7	0.30 ± 0.5
Seaquest	73.00 ± 26.3	14.00 ± 15.6
Skiing	−1248.00 ± 0.0	−1248.00 ± 0.0
Solaris	0.00 ± 0.0	0.00 ± 0.0
SpaceInvaders	130.75 ± 29.5	41.00 ± 25.6
StarGunner	370.00 ± 95.4	60.00 ± 106.8
Tennis	0.10 ± 0.7	−1.90 ± 0.9
TimePilot	370.00 ± 134.5	105.00 ± 66.9
Tutankham	17.85 ± 11.8	1.95 ± 4.8
UpNDown	1247.50 ± 465.4	272.00 ± 285.5
Venture	0.00 ± 0.0	0.00 ± 0.0
VideoPinball	5026.10 ± 1909.3	521.10 ± 365.4
WizardOfWor	240.00 ± 156.2	30.00 ± 55.7
YarsRevenge	5527.45 ± 2895.4	274.10 ± 315.1
Zaxxon	20.00 ± 60.0	0.00 ± 0.0

Additional Implementation Details

The code along with the experiment result files are available at https://github.com/stefanotoole/N-CPL.

Table 11: N-CPL hyperparameters.

Lookahed Parameters	
Sim. Interactions Budget	100
Lookahead Horizon	100
Value and Policy Network Parameters	
Batch size	128
Learning Rate	2.50×10^{-4}
Epochs	8
Loss Function for Policy	Categorical crossentropy
Loss Function for Value Function	Huber
Discount factor used in TD Learning	0.99
Time steps between target network updates (for value network)	10,000
Interval size of learning schedule	1×10^6 sim. interactions

Table 11 shows the different hyperparameters of N-CPL along with the selected values used for the experiments. Due to computational restraints we could not tune the hyperparameters of N-CPL. For the lookahead parameters the simulator budget per time step was selected to be 100 to match the budget used by Junyent et al. (10). The lookahead horizon is the maximum search depth allowed, that is the maximum number of actions allowed from the root node of the lookahead. For the value and policy network parameters both the batch size and number of epochs used affects the training time. We selected both the batch size and number of epochs such that the time spent training the networks using a single vCPU is around the same time as the planning steps of N-CPL. We used the same learning rate, loss function (for the value function) and discount factor for the TD learning as used by DQN (2). Following Junyent et al.(8) we use a crossentropy loss function for the policy network. The interval size of the learning schedule dictates the size of the data set used to update the networks and how often to reject or accept parameter updates. The interval size of the learning schedule is illustrated in Figure 2 of the main paper to be of a size of N episodes. Instead of setting the interval size to be a set number of episodes we set it as 1×10^6 simulator interactions.